
- Going back to your scenarios, you've been asked to identify what areas might have security risks and what your plan is to address them
  - What type of Compliance issues would you need to deal with?
  - Identify 2-3 potential security issues/threats that might be associated with your scenario
  - What features of IAM could help with your scenario?
  - What other types of checks or precautions would you recommend to protect against these threats?

# RECAP

- What is the AWS Shared Responsibility Model?
  A. AWS is responsible for all security
  B. Customers are responsible for data center security
  C. AWS handles cloud security; customers secure their applications ✅
  D. Customers handle physical security only
- What is an IAM Role used for?
  A. Storing access logs
  B. Encrypting S3 buckets
  C. Assigning temporary permissions to services or users ✅
  D. Creating root users
- AWS is responsible for configuring your virtual firewalls and access policies
  - True
  - False ✅

RECAP



- AWS is responsible for security of the cloud; the customer is responsible for security in the cloud

| You | AWS |
| --- | --- |
| Implementing access management that restricts access to AWS resources like S3 and EC2 to a minimum, using AWS IAM | Protecting the network through automated monitoring systems and robust internet access, to prevent Distributed Denial of Service (DDoS) attacks |
| Encrypting data at rest (e.g. database or other storage systems), network traffic to prevent attackers from reading or manipulating data (for example, using HTTPS) | Performing background checks on employees who have access to sensitive areas |
| Configuring a firewall for your virtual network that controls incoming and out- going traffic with security groups and Access Control Lists (ACLs) | Decommissioning storage devices by physically destroying them after end of life |
| Managing patches for the OS and additional software on virtual machines | Ensuring the physical and environmental security of data centers, including fire protection and security staff |

# RECAP




- Understanding Compliance (HIPPA, SOX, GDPR, PCI, etc.)
  - Where in the world is the data center?
  - How do you enforce access controls?
  - How are you protecting the data?
- Managing access to AWS services and resources securely with Identity and Access Management (IAM)
  - IAM is an AWS service that helps securely control access to AWS resources
- Protecting Data with Encryption
  - To encrypt data, you need a key to start an encryption and to decrypt the message
  - If someone is listening and hijacks the data, they can't read it because they don't have the proper keys to unlock the message

# Disaster Recovery & Resiliency in the Cloud

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

# Where we are at…



- To this point, we've created some pretty simple applications
- When we develop applications on the cloud, it's important to ensure they are running as we expected with minimal downtime
- An outage of any length can have negative impacts and be very costly
- But if we are running in the cloud, nothing could possibly go wrong, right?!?
- Think again…

# Recent Cloud Outages – MS Azure (2021)




- Microsoft Corp. was hit by a massive cloud outage today that took most of its internet services offline
- Microsoft's Azure cloud services, as well as Teams, Office 365, OneDrive, Skype, Xbox Live and Bing, were all inaccessible due to the outage
- Even the Azure Status page was reportedly taken offline
- Microsoft blamed that issue on "a recent change to an authentication system"

# Recent Cloud Outages – Google (2019)




- A configuration change intended for a small group of servers in one region was wrongly applied to a larger number of servers across several neighboring regions.
- This caused a 10 percent drop in YouTube traffic and a 30 percent drop in Google Cloud Storage traffic. The incident also impacted one percent of more than one billion Gmail users.
- Big name customers that were affected include Snapchat, Vimeo, Shopify, Discord, and Pokemon GO

# Recent Cloud Outages – AWS (2017)

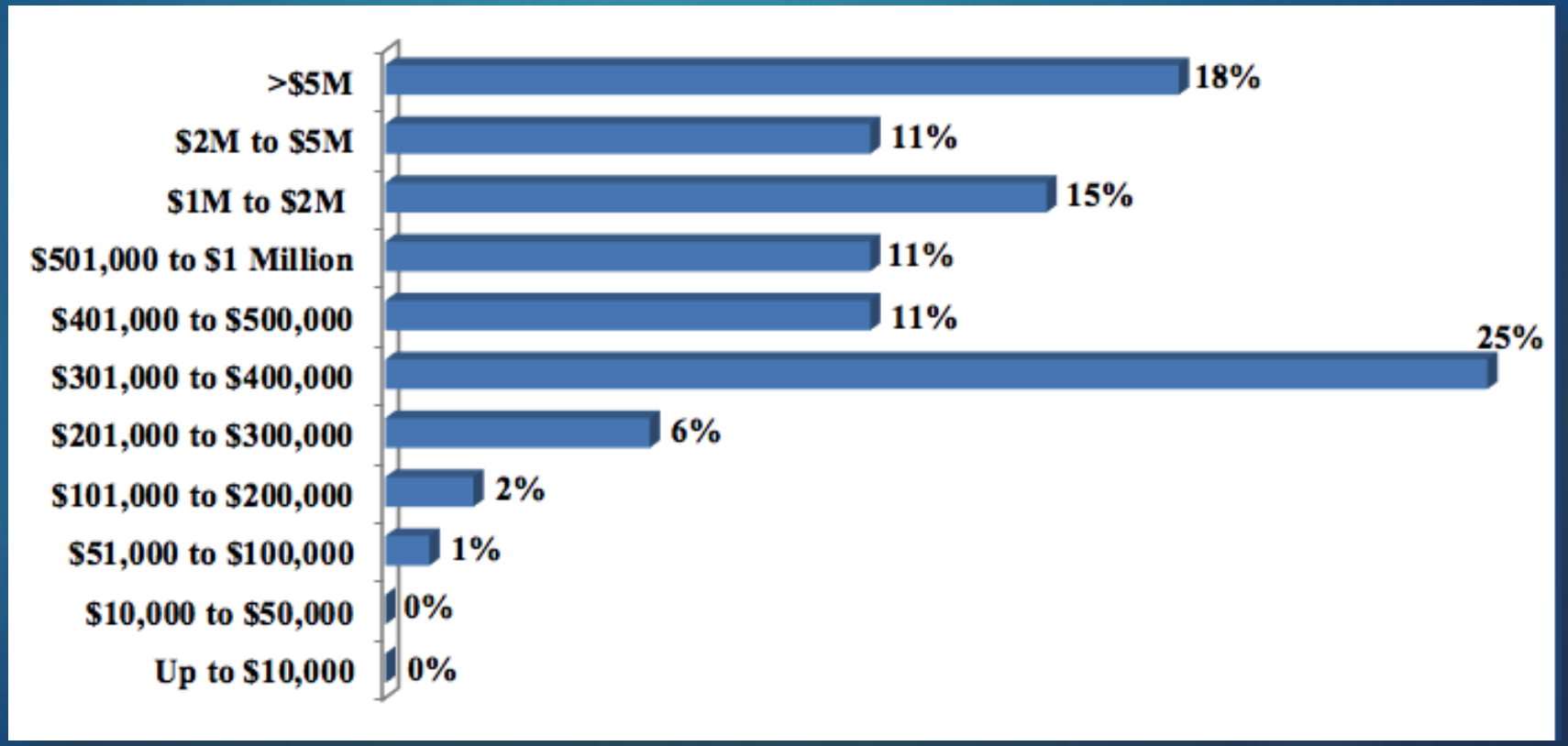



- *"This was the outage that shook the industry"*
- An Amazon Web Service engineer trying to debug an S3 storage system in the Virginia data center accidentally typed a command incorrectly and much of the Internet, including many enterprises like Slack, Quora and Trello were down for 4 hours

# Downtime can be really expensive…



- In 2021, 91% organizations said a single hour of downtime that takes mission-critical server hardware and applications offline
- Averages over $300,000 due to lost business, productivity disruptions and remediation efforts

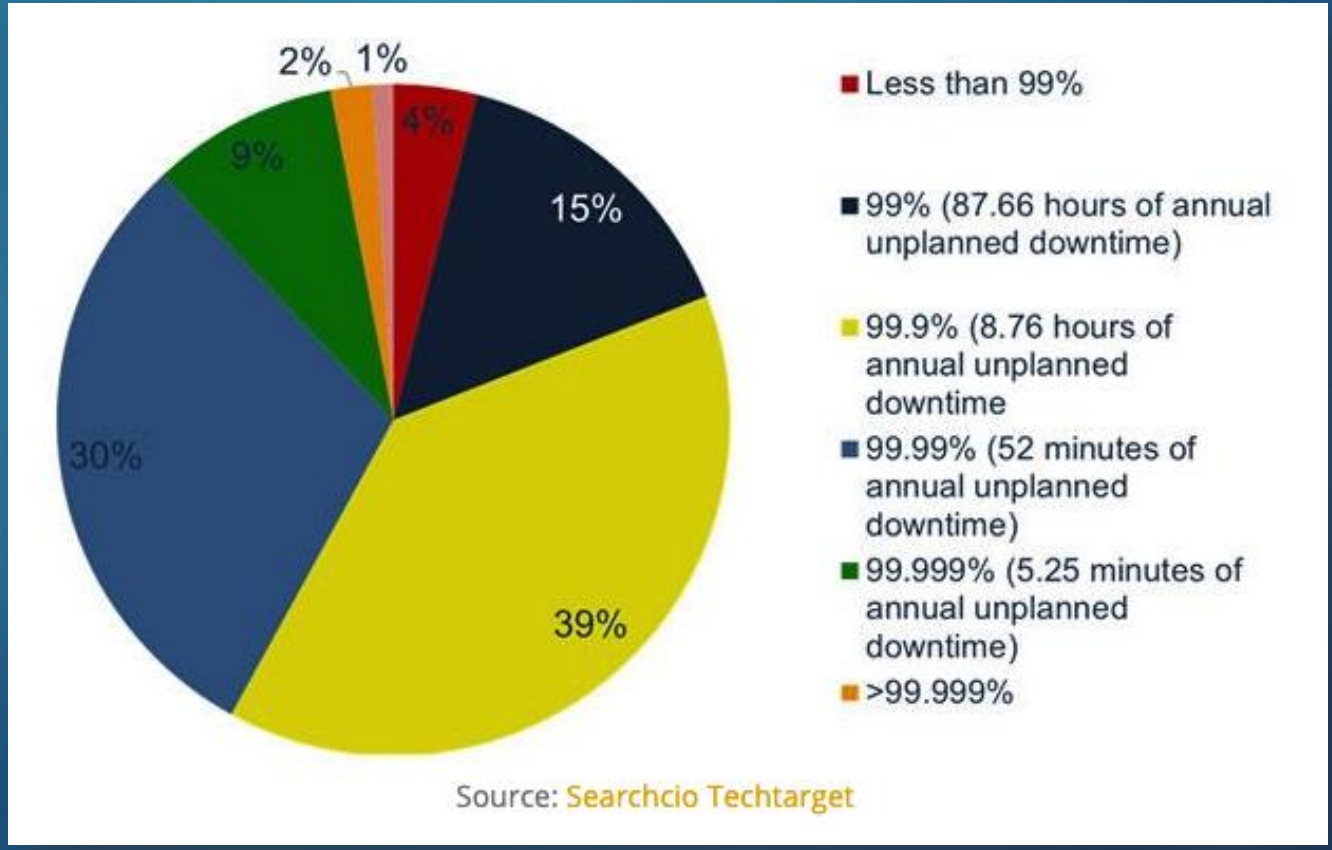



Source: https://techchannel.com/IT-Strategy/09/2021/cost-enterprise-downtime

# …and companies have high standards



According to another survey, while companies can't achieve zero downtime, one in every 10 companies said that their availability must be greater than 99.999%.



# Business Impacts of Downtime



- Business disruption
  - The impact of downtime varies according to the application and the business
- Loss of revenue/sales and profitability
  - The longer the downtime, the more impact this will have
- Damaged to reputation
  - In just a few minutes, word about a "lengthy" downtime can reach across the globe via social media
  - A company's inability to meet expectations and SLAs can turn potential customers away and drive stock down
- Loss of proprietary and critical data
  - The potential loss of data due to a system outage can have significant legal and financial implications

# RPO vs RTO



- Recovery Point Objective (RPO) and Recovery Time Objective (RTO) are two of the most critical parameters of a data protection plan and disaster recovery strategy
- Despite their similarities, RPO and RTO serve different purposes and come with different metrics:
  - Recovery Point Objective = Data Risk. RPO refers to the maximum acceptable amount of data loss an application can undergo before causing measurable harm to the business


  - Recovery Time Objective = Downtime. RTO states how much downtime an application experiences before there is a measurable business loss
  - For mission-critical applications, minimizing risk may require zero/near–zero RPOs and RTOs – despite the expense

Source: https://www.keepitsafe.com/blog/post/the-difference-between-rto-and-rpo/

# Downtime – What to do?



- Downtime is inevitable
- Just because you have a system on the cloud, ignoring the likelihood of any sort downtime would be foolish (and potentially very costly)
- Prevention is the key for avoiding unplanned downtime and reducing its impact
- Some ways to minimize downtime include:
  - Planning for Disaster Recovery
  - Building Resilient applications
  - Having a Deployment Strategy that minimizes downtime
  - Thorough testing for potential cloud outage scenarios

# Disaster Recovery



- Disaster Recovery is the function of restoring normal operations after an unplanned event occurs
- How does it work?
  - Disaster recovery relies upon the replication of data and computer processing in an off-premises location not affected by the disaster
  - When servers go down because of a natural disaster, equipment failure or cyber attack, a business needs to recover lost data from a second location where the data is backed up
  - Ideally, an organization can transfer its computer processing to that remote location as well in order to continue operations
- How would this work on the cloud?


Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Disaster Recovery - AWS

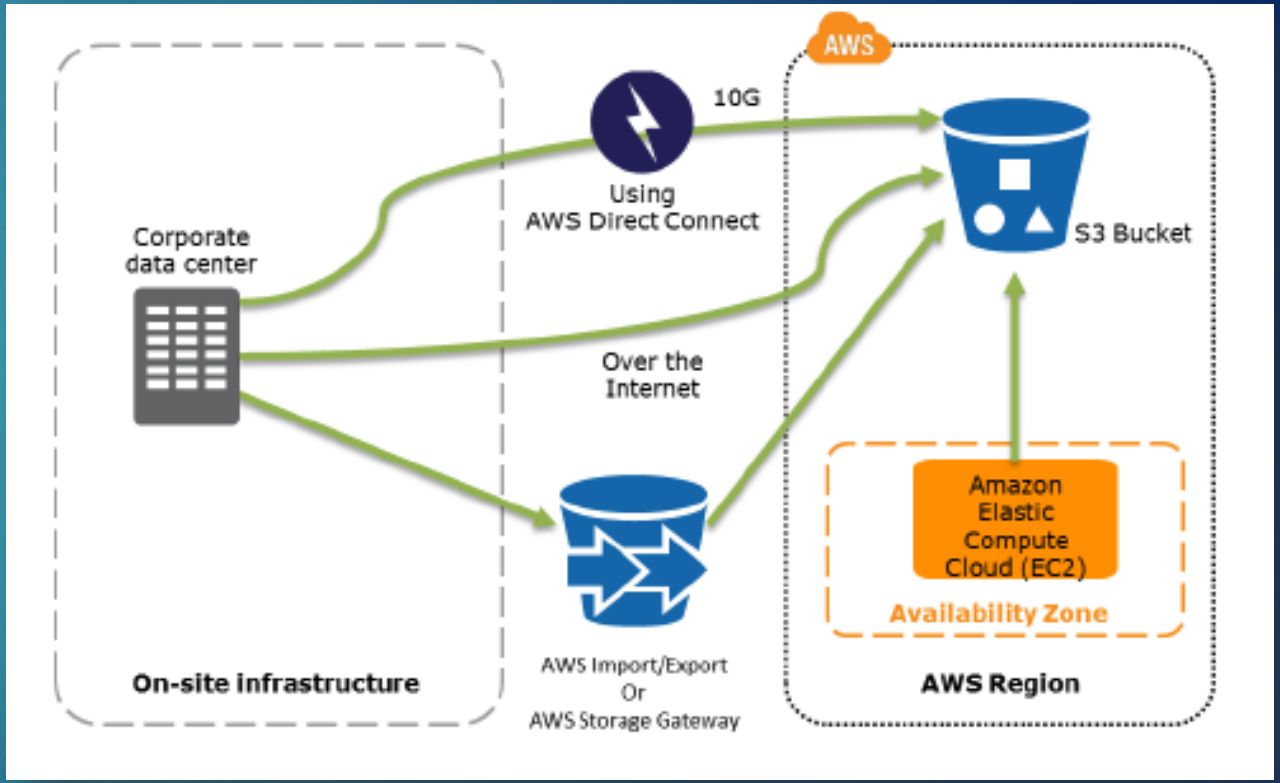


- With AWS, there are a few options to consider, based on accepted risk and associated costs:
  1. Traditional Backup and Restore
  2. Pilot Light for Quick Recovery (Active - Cold Backup)
  3. Warm Standby
  4. Multi-site Solution (Active – Active)
- These can apply to either:
  - Data Center ⇔ AWS Region
  - AWS Region ⇔ AWS Secondary Region

# Backup and Restore



- In most traditional environments, data is backed up to tape and sent off-site regularly
- Amazon S3 is an ideal destination for backup data, as it is designed to provide 99.999999999% reliability
- For systems running on AWS, customers also back up into Amazon S3

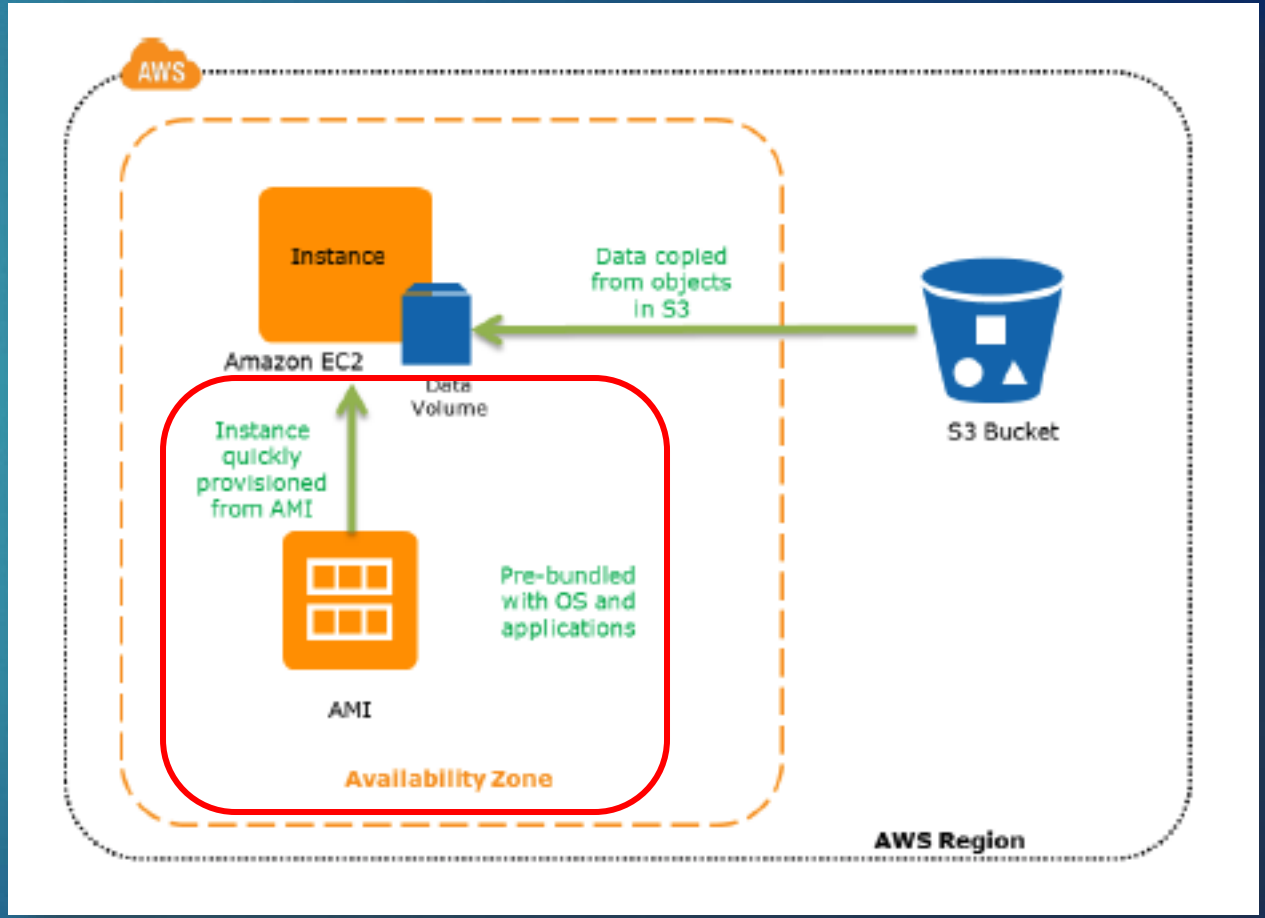



Source: https://www.ecloudgate.com/Doc/DisasterRecovery_Overview

# Backup and Restore



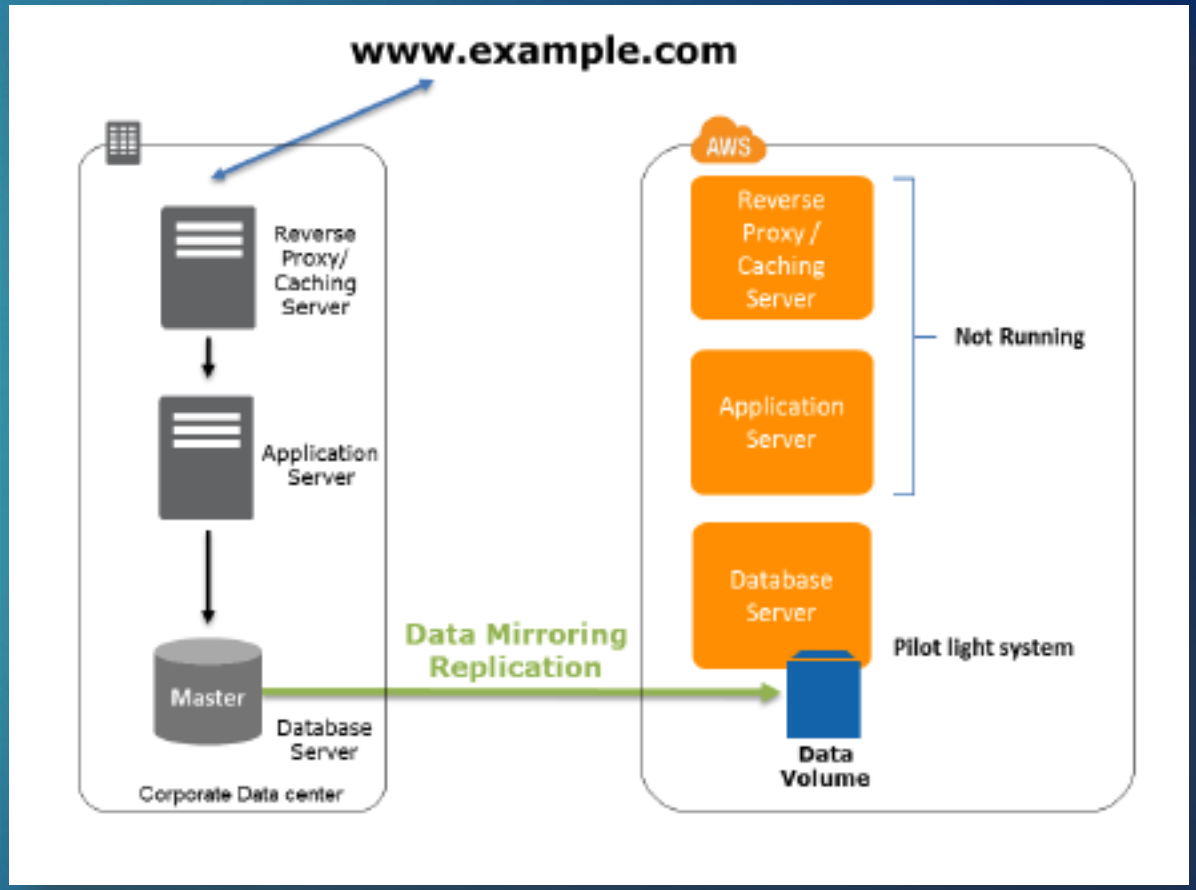
- Backup of your data is only half the story
- Recovery of data in a disaster scenario needs to be tested and achieved quickly and reliably
- EC2 instances can be quickly provisioned from Amazon Machine Images (AMI), which have been pre-configured with the proper OS and applications



# Pilot Light



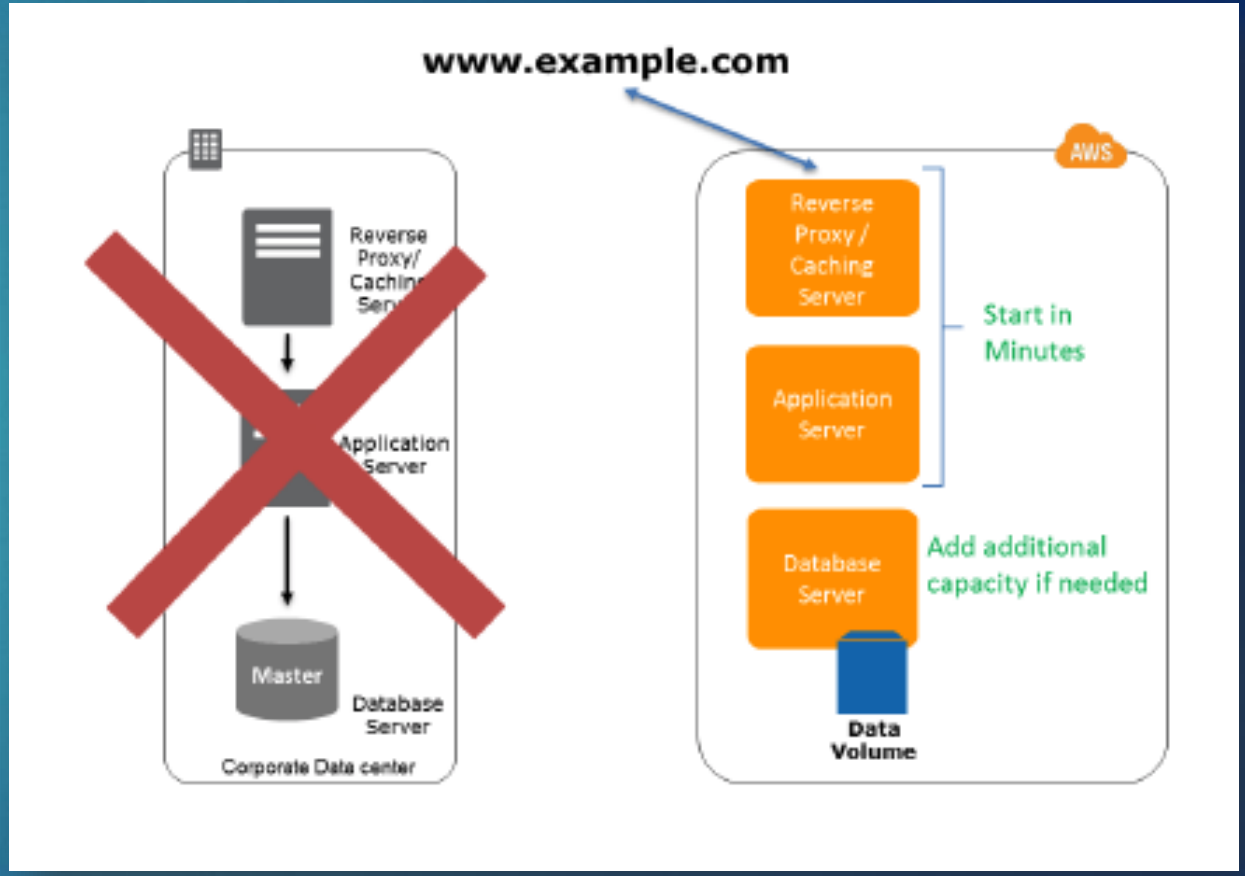
- Uses the analogy that a small idle flame (pilot light) that's always on can quickly ignite the entire furnace to heat up a house as needed
- You must ensure that you have the most critical core elements of your system already configured and running in AWS (the pilot light)
- When the time comes for recovery, you would then rapidly provision a full scale production environment around the critical core



# Pilot Light - Recovery



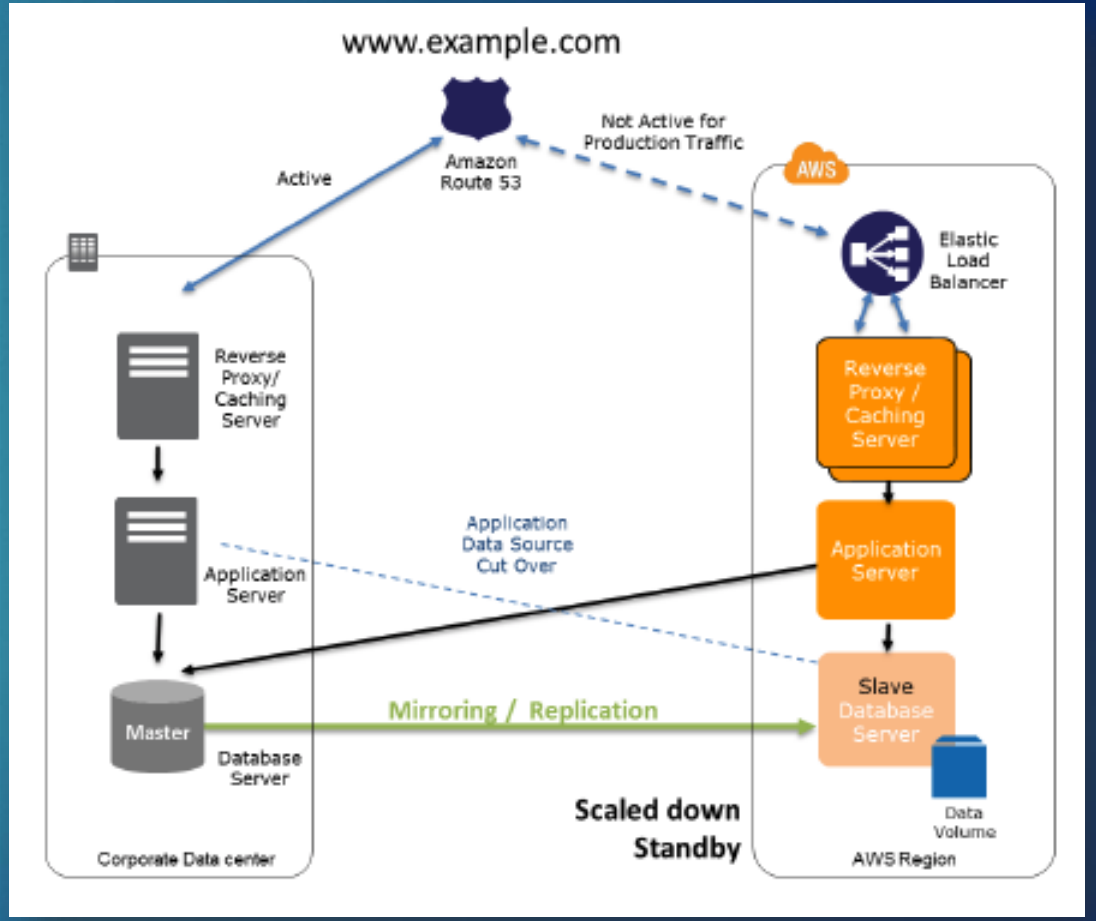
- To recover the remainder of the environment around the pilot light, you would start your systems from the Amazon Machine Images (AMIs) in minutes on the appropriate instance types
- Start your application EC2 instances from your custom AMIs
- Resize and/or scale any database / data store instances, where necessary
- Change DNS to point at the EC2 servers



# Warm Standby



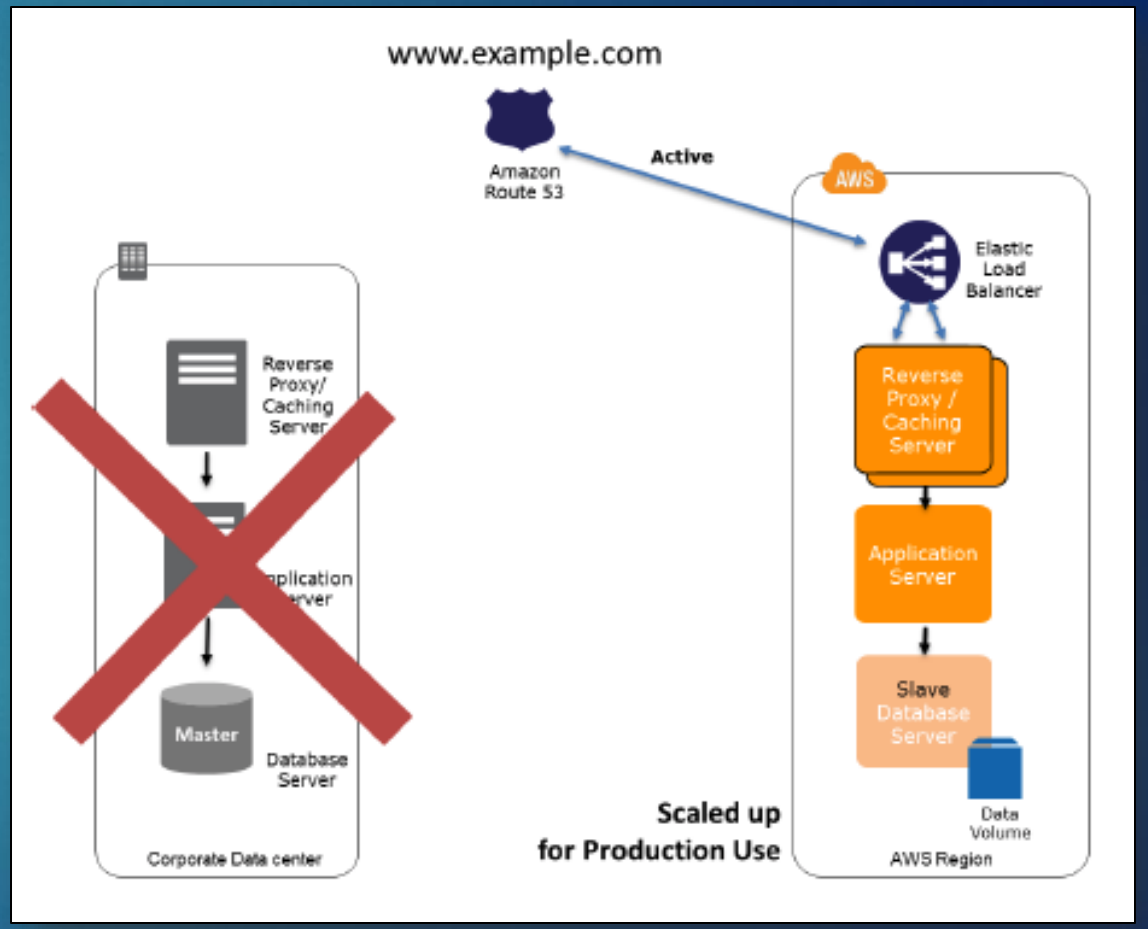
- A warm standby solution extends the pilot light elements and preparation
- It further decreases the recovery time because in this case, some services are always running
- These servers can be running on a minimum sized fleet of EC2 instances on the smallest sizes possible
- This solution is not scaled to take a full-production load, but it is fully functional



# Warm Standby - Recovery



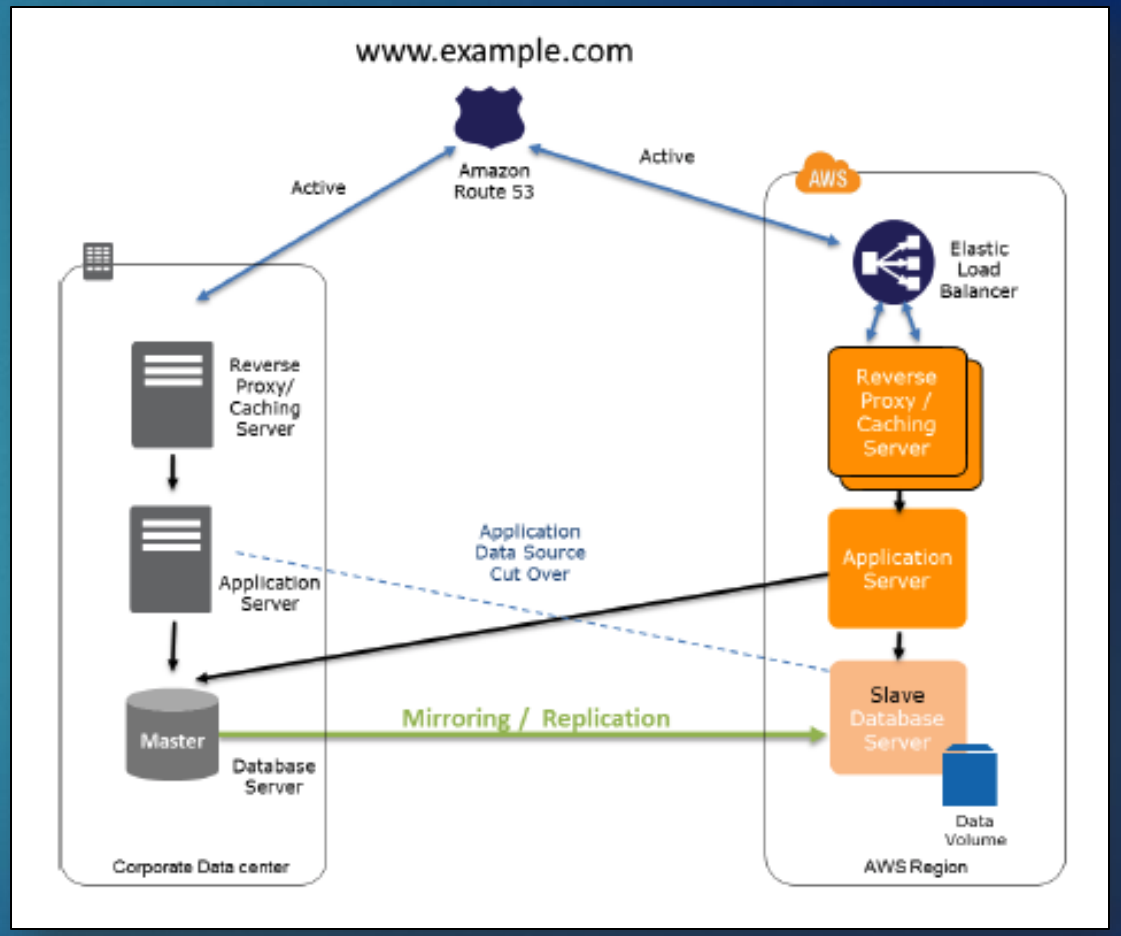
- In the case of failure of the production system, the standby environment will be quickly scaled up for production load
- DNS records are changed to route all traffic to the AWS Region



# Multi-site Solution



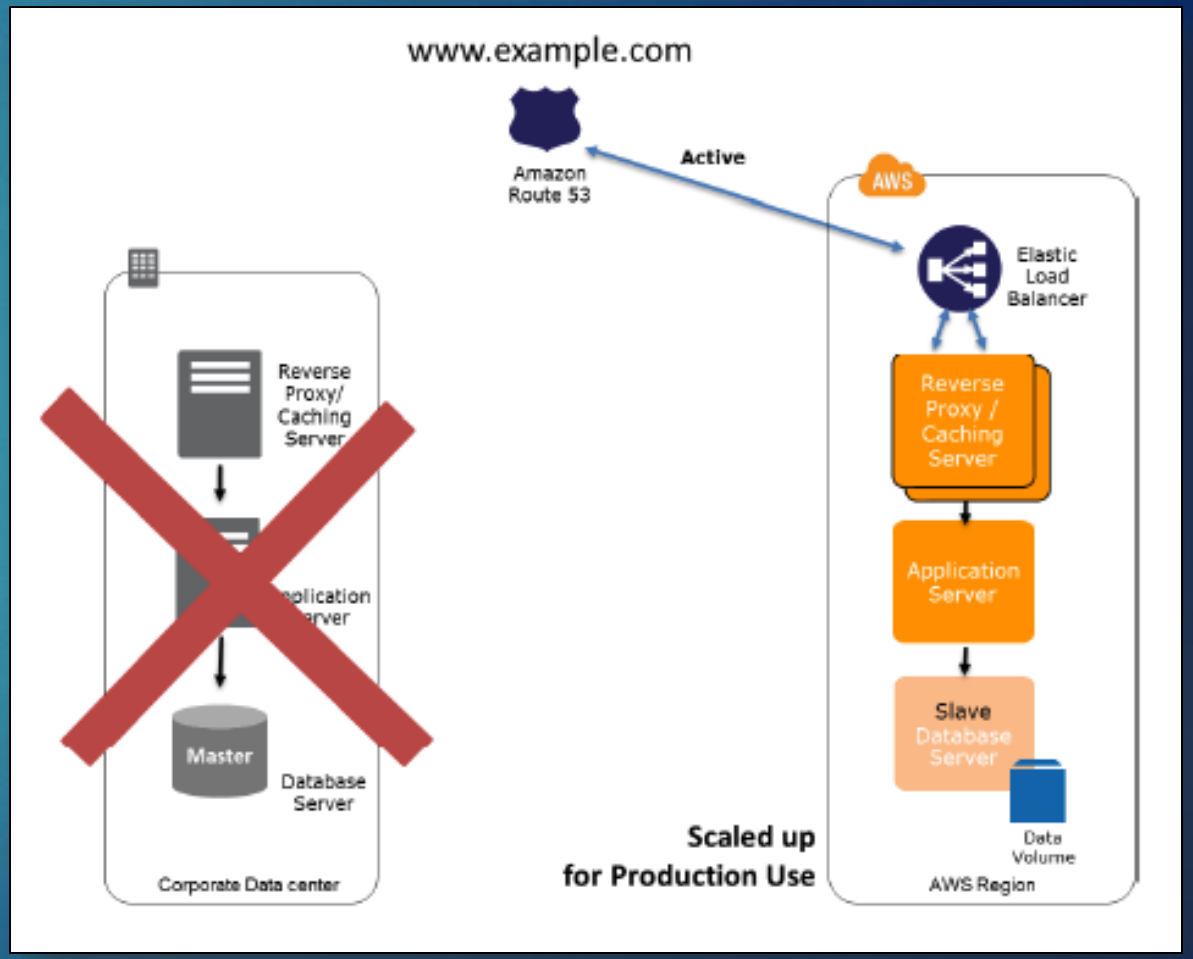
- A multi-site solution runs in AWS as well as on your existing on-site infrastructure (or another AWS Region)
- Commonly referred to as Active-Active configuration as the workload is balanced to one or more locations



# Multi-site Solution - Recovery



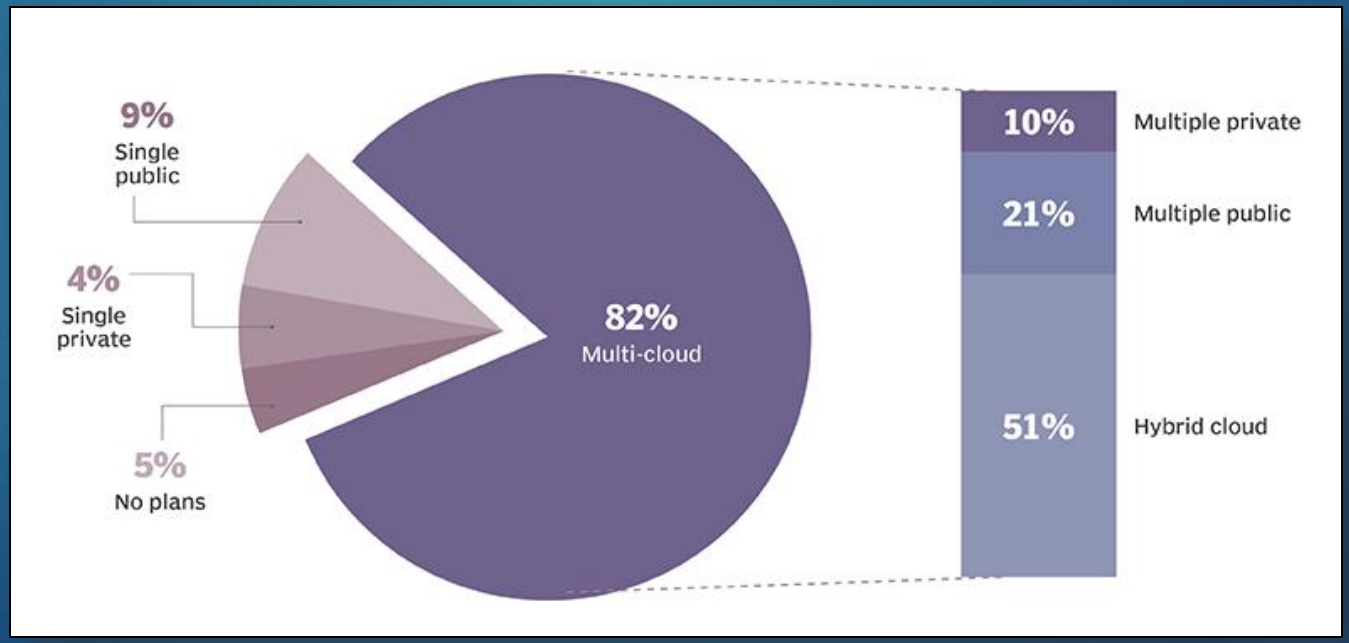
- Traffic is cut over to the AWS infrastructure by updating DNS



# Multi-cloud Disaster Recovery



- More organizations are moving towards a multi-cloud solution for DR to minimize impacts from an outage occurring with a single cloud-provider
- This requires more coordination and testing but may warrant the investment for organizations that cannot afford any unforeseen outages (e.g. earthquakes)

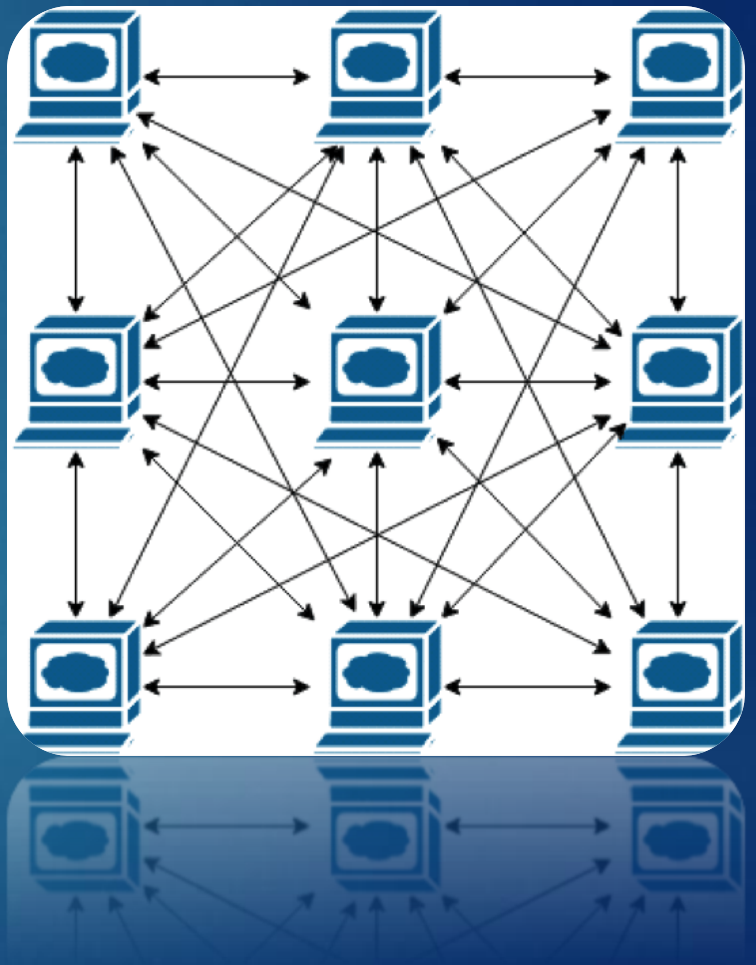



Source: https://searchdisasterrecovery.techtarget.com/tip/5-crucial-multi-cloud-disaster-recovery-principles

# Resiliency



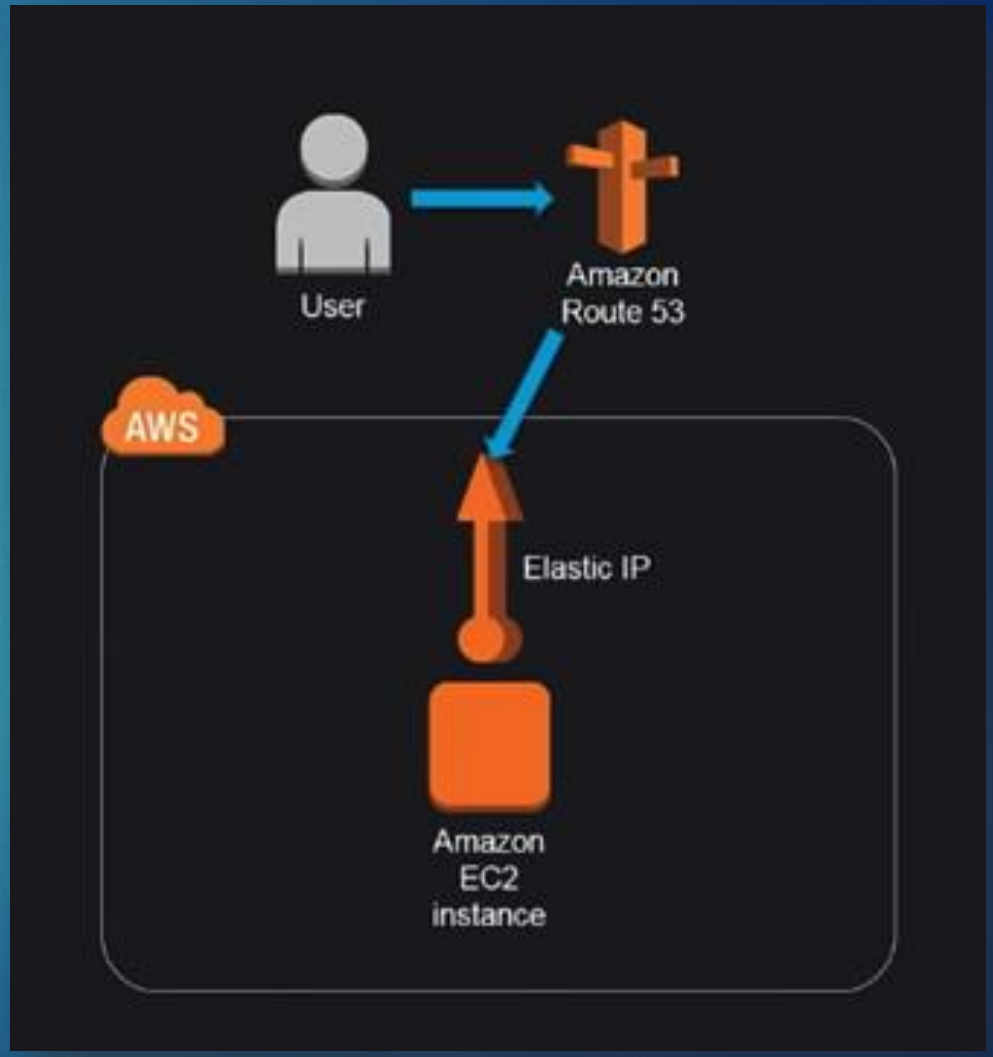
- Resiliency is about building an architecture that can withstand the loss of a component with minimal (or no) impact
- While disaster recovery focuses on recovering from a disaster, IT resilience focuses on the proactive measures that a business can put in place to keep running even when the worst comes knocking
- A resilient workload not only recovers, but recovers in an amount of time that is desired as stated by the Recovery Time Objective (RTO)



# Resiliency – Example



- This very similar to what we did with our first WordPress example
- You have a single instance (Amazon EC2)
- You attach an Elastic IP address to it
- You can also put an entry in AWS managed DNS service (Route 53)
- This works great for one user
- Problem?
  - There's no failover (resiliency)
  - No disaster recovery

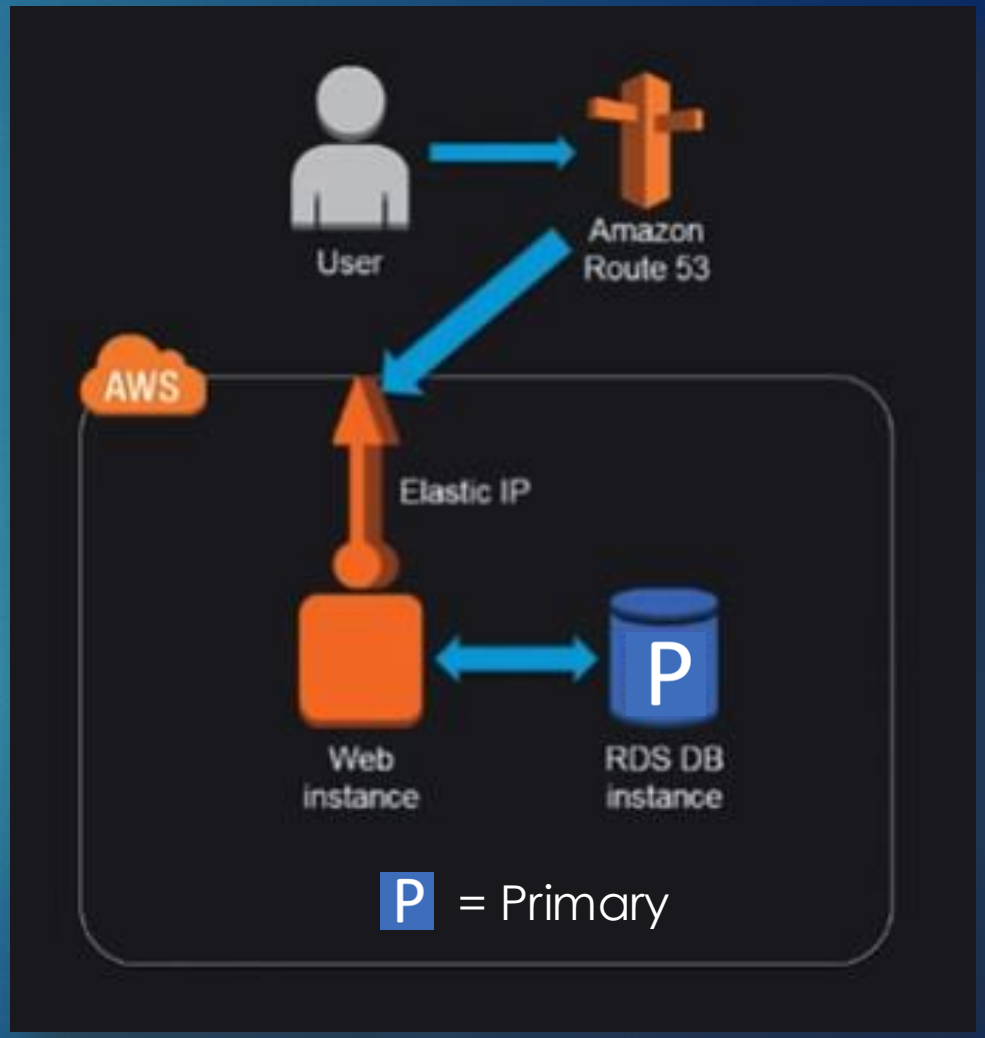



Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Resiliency – Example



- Separating the web server from the database will allow these to scale separately
- Adding Auto-scaling to add an instance if one goes down
- Using a fully managed service for the database will ensure data is not lost
  - AWS takes care of provisioning, patching, configuration, backups, or recovery
- Problem?
  - Scalability may be a problem, which can overwhelm servers
  - Still no disaster recovery

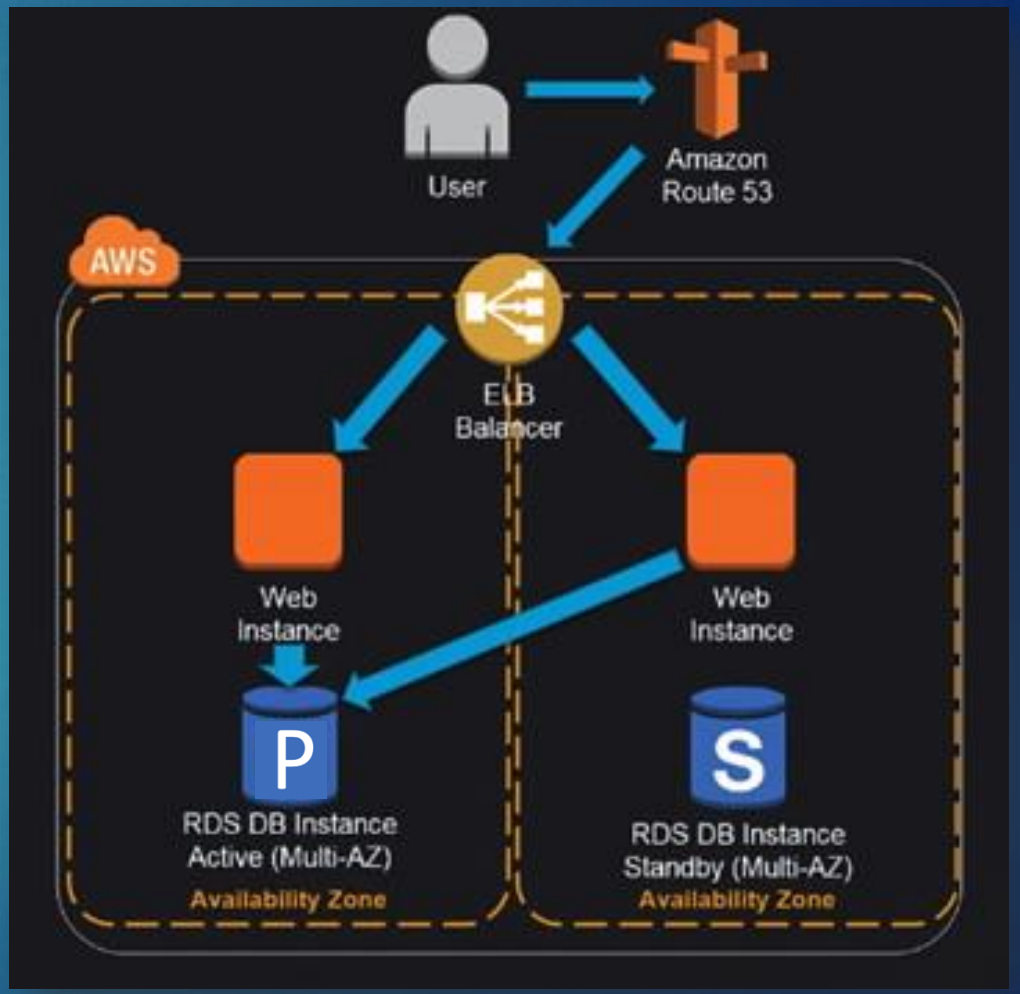



Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Resiliency – Example



- As more users there are on the web app, the more failover and redundancy issues arise
- The first thing to do is to add a new web instance in a second availability zone
  - The latency between these two availability zones is a few milliseconds
- Elastic Load Balancing (ELB) enables the users to be divided between web instance 1 or 2, according to the traffic in each of the Availability Zones
  - If one of the web servers becomes unavailable, the Load Balancer redirects the traffic to the other web server
- Problem
  - Scalability could be better

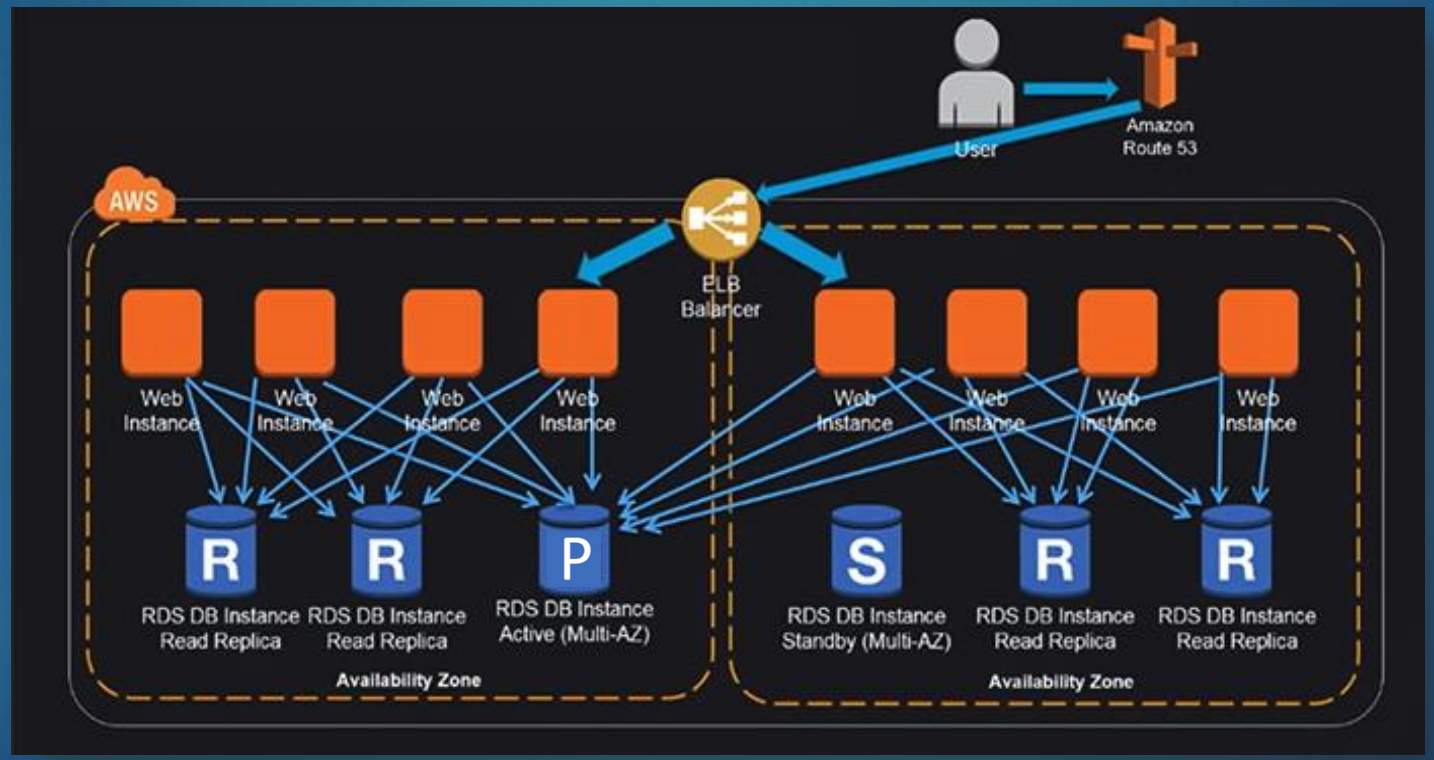



Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Resiliency – Example



- By adding a multitude of availability zones and web instances within each, it is possible to accommodate more than 100,000 concurrent users on your application



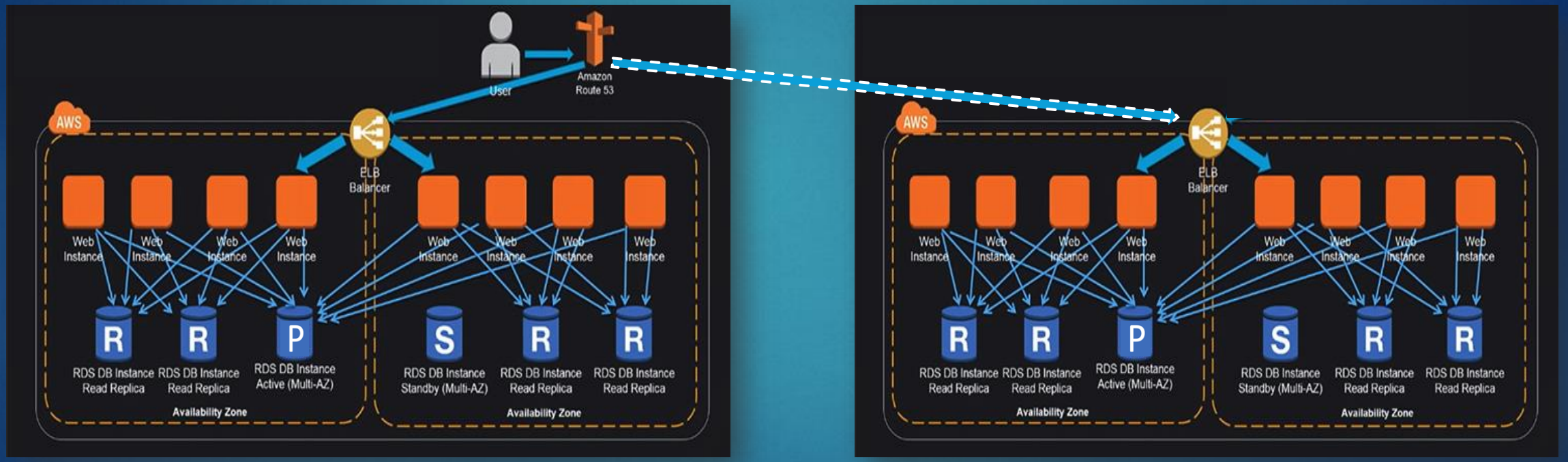

- What about Disaster Recovery?

Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Resiliency & Disaster Recovery



- Adding another region will provide a DR backup if all the availability zone completely goes down




Region #1

(Primary)

Region #2

(Disaster Recovery)

Source: https://www.artifakt.io/blog/2018/01/news/cto-advice-scalability-resilience-with-aws-7599

# Resiliency – Recap



- A resilient workload has the capability to recover when stressed by load (more requests for service), attacks (either accidental through a bug, or deliberate through intention), and failure of any component in the workload's components
- A resilient workload not only recovers, but recovers in an amount of time that is desired (Recovery Time Objective – RTO)
- To minimize human intervention, consider implementing automation to:
  - Replace redundant components automatically when they fail
  - Fail over to backup components when the primary component fails
  - Restart components that cannot be made redundant or fail over

# Deployment Challenges

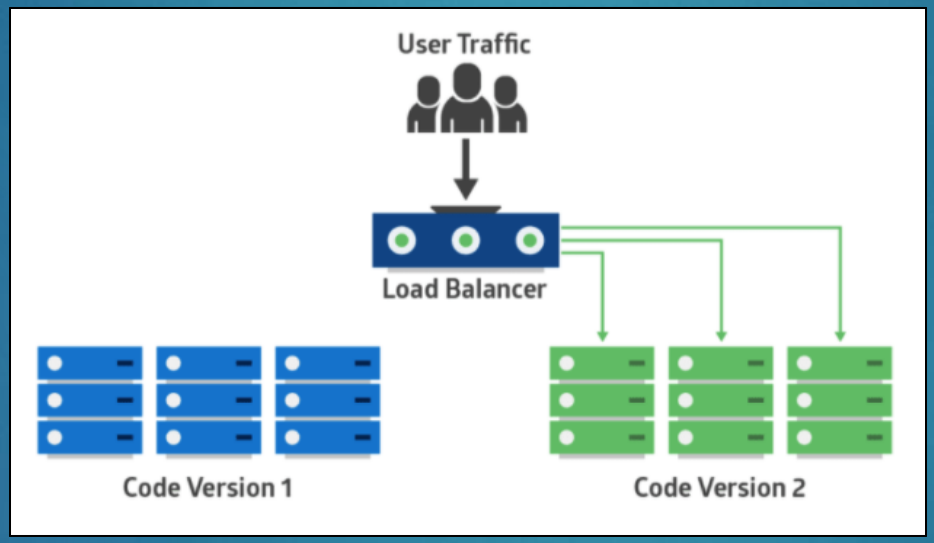


- These days, the biggest change to software development is the frequency of deployments
- Product teams deploy releases to production earlier (and more often)
- Moving to a microservices architecture makes this even more challenging
- Many organizations will wait to deploy off hours  to push the updates to the production environment (when the least amount of users are active)
- CI/CD helps to alleviate some of these challenges using automation, but is there a way to deploy with no downtime?
- 3 deployment strategies to consider:
  - Blue-Green
  - Canary
  - Rolling

# Deployment Strategy – Blue-Green



- Two identical production environments work in parallel
- One is the currently-running production environment receiving all user traffic (Blue)
- The other is a clone of it, but idle (Green)



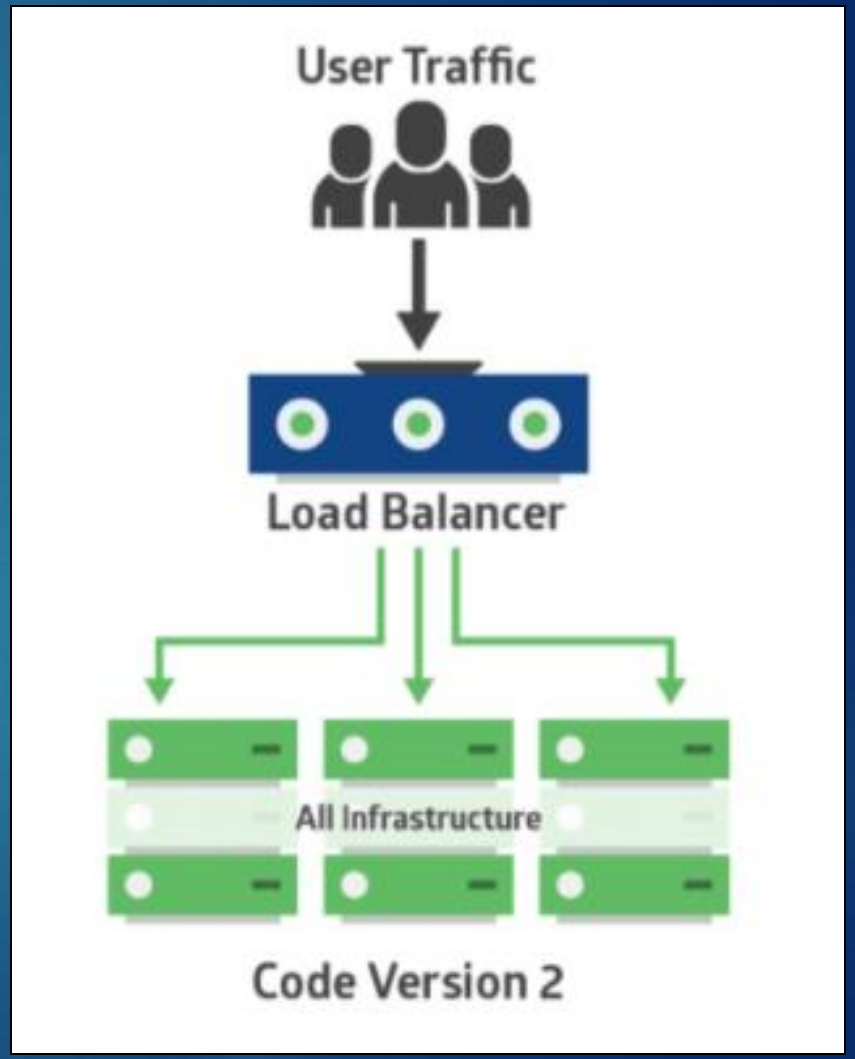

- The new version of the application is deployed in the Green environment and tested for functionality and performance
- Once the testing results are successful, application traffic is routed from Blue to Green
- Green then becomes the new production

Source: https://dev.to/mostlyjason/intro-to-deployment-strategies-blue-green-canary-and-more-3a3

# Deployment Strategy – Canary



- Canary deployment is like Blue-Green, except it's more risk-averse
- Instead of switching from Blue to Green in one step, you use a phased approach
- Once the application is signed off for release, only a few users are routed to it to minimize any impact
- With no errors reported, the new version can gradually roll out to the rest of the infrastructure

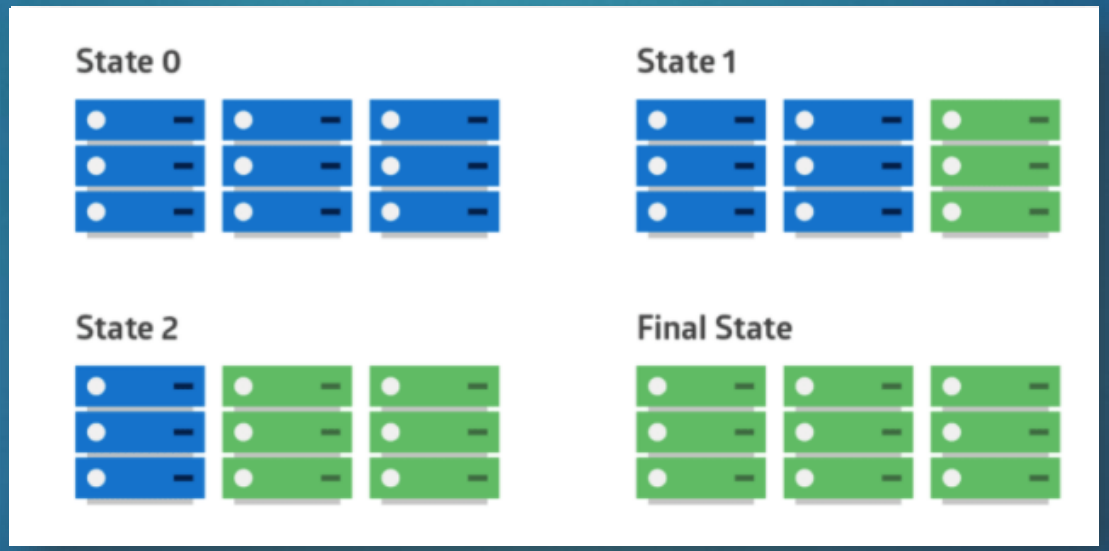



Source: https://dev.to/mostlyjason/intro-to-deployment-strategies-blue-green-canary-and-more-3a3

# Deployment Strategy – Rolling



- Rolling Deployment is similar to Canary but the difference is we update the newer version into a small number of instances
- An application's new version gradually replaces the old one over a period of time




- During this time, new and old versions will coexist without affecting functionality or user experience
- This process makes it easier to roll back any new component incompatible with the old components

Source: https://dev.to/mostlyjason/intro-to-deployment-strategies-blue-green-canary-and-more-3a3

# Deployment Challenges – Which is best?



- The answer depends on the type of application you have and your target environment
- Blue/Green
  - Rollback is easy as we have a complete copy of the production environments
  - Easy rollbacks in case of failure
- Canary
  - No downtime while updating or releasing the new feature
  - Cheaper in terms of Infrastructural resources
- Rolling
  - The rollback process is very easy as we are deploying an instance by instance
  - Cost-effective as the effort and infrastructure utilization is highly less

# Testing for cloud outage scenarios



- Error handling code and failure handling procedures are usually the least well-tested aspects of an application
- So how do you know you have built a reliable system?
- You can't ask a giant cloud provider just to switch OFF a building to see if your app remains available for your customers
- If you know that things will fail, you can build mechanisms to ensure your system persists no matter what happens

# Chaos Engineering – Break and Destroy

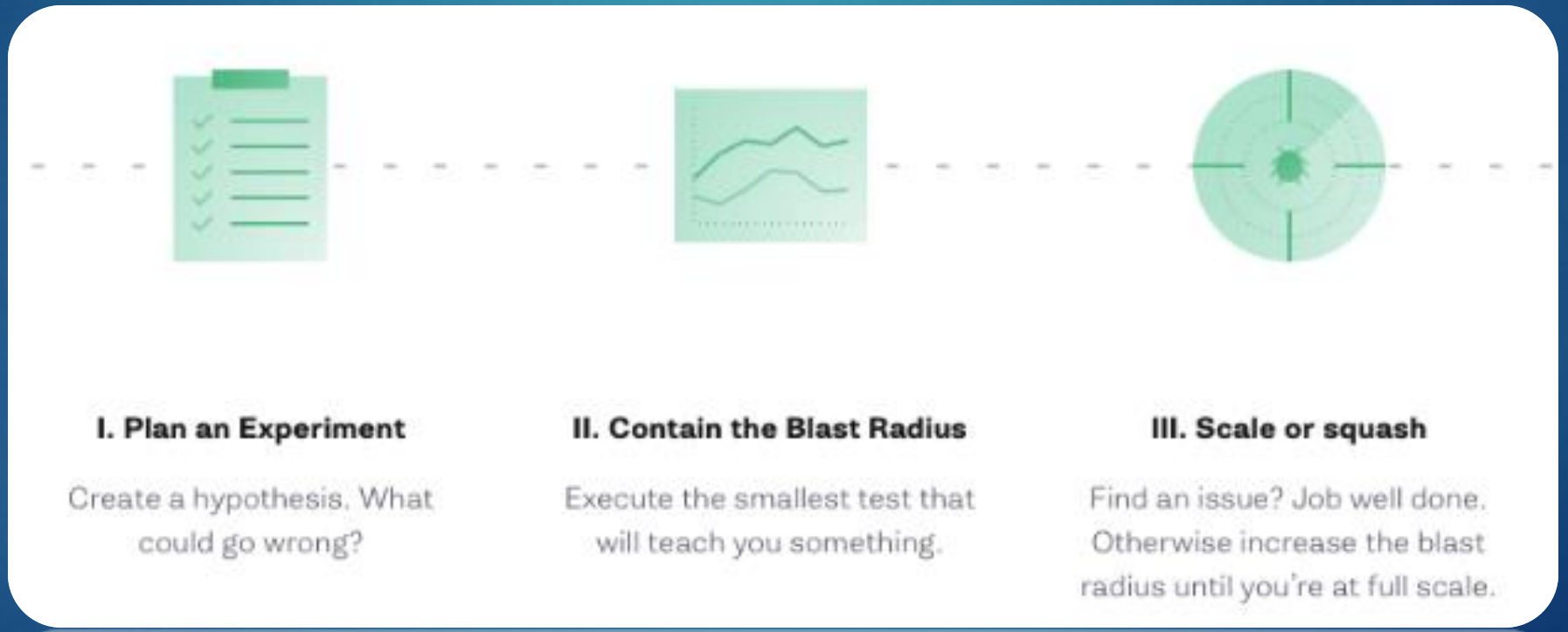


- Chaos Engineering is a specific discipline that allows engineers to test the worst possible case scenarios against cloud distributed systems built from multiple services to find its resiliency against such events
- It can be used to achieve resilience against:
  - Infrastructure failures
  - Network failures
  - Application failures

# Chaos Engineering – Break and Destroy



- Chaos Engineering involves running thoughtful, planned experiments that teach us how our systems behave in the face of failure
- These experiments generally follow three steps:

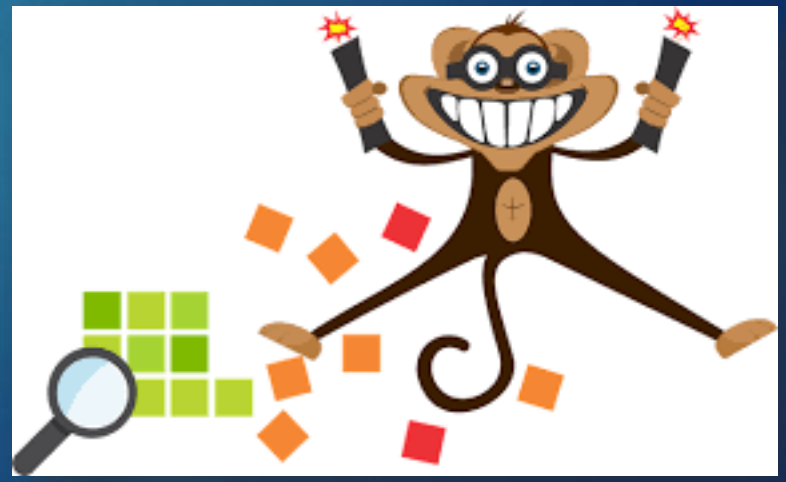



Source: https://www.gremlin.com/community/tutorials/chaos-engineering-the-history-principles-and-practice/

# Chaos Engineering - Examples



- Simulating the failure of an entire region or datacenter
- Injecting latency between services for a select percentage of traffic over a predetermined period of time
- Function-based chaos (runtime injection): Randomly causing functions to throw exceptions
- Code insertion: Adding instructions to the target program and allowing fault injection to occur prior to certain instructions
- Time travel: Forcing system clocks out of sync with each other
- Executing a routine in driver code emulating I/O errors
- Maxing out CPU cores on a cluster



Source: Chaos Engineering: Building Confidence in System Behavior through Experiments

# Chaos Engineering – Tools



- Chaos Monkey (Netflix)
  - Randomly terminate instances in production to ensure that engineers implement their services to be resilient to instance failure
- Simian Army (Netflix)
  - Suite of failure-inducing tools designed to add more capabilities beyond Chaos Monkey
  - Latency Monkey - Introduces communication delays to simulate degradation or outages in a network
  - Chaos Gorilla – Disables an entire Availability Zone
  - Chaos Kong – Disable an entire Region
- Gremlin
  - First hosted chaos engineering service (Failure as a Service)




- From the previous scenario exercise, you provided with the AWS Cost Calculator and Auto-scaling, you have been asked by the organization to explain your recommendations for DR & Resiliency. You have been tasked to identify the following:
  - What is your recommended Recovery Time Objective (RTO) and Recovery Point Objective (RPO)?
  - What would be necessary for your project to ensure a solid DR plan?
  - What would you do to ensure your application is resilient?
  - Would you use certain managed services, deployment strategies or other precautions to avoid an outage?
  - What type of "chaos engineering" techniques would you use to test resiliency?
  - You have 15-20 minutes to discuss your recommended plan
- Download the template from Assignments > Activity - DR & Resiliency Recommendations
  - 1 submission per team
- The scenarios can be found in Assignment > Activity - Cost Estimating Scenarios